Image Processing

ECSE 313 Group 16 Lab 5 Report

Nathaniel Hahn and Donovan Crowley

April 22, 2026

Contents

1 Introduction Exercise **2**

2 Histogram of an Image **4**

3 Pointwise Transformations **5**

4 Gamma Correction **8**

5 Image Smoothing **11**

6 Image Sharpening **13**

1 Introduction Exercise

By converting the image pixels from uint8 to double, the provided 'yacht.tif' pixels can be adjusted to invert, negate, and multiply the image.

Figure 1: Original Yacht Image

Figure 2: Horizontally Flipped Yacht Image

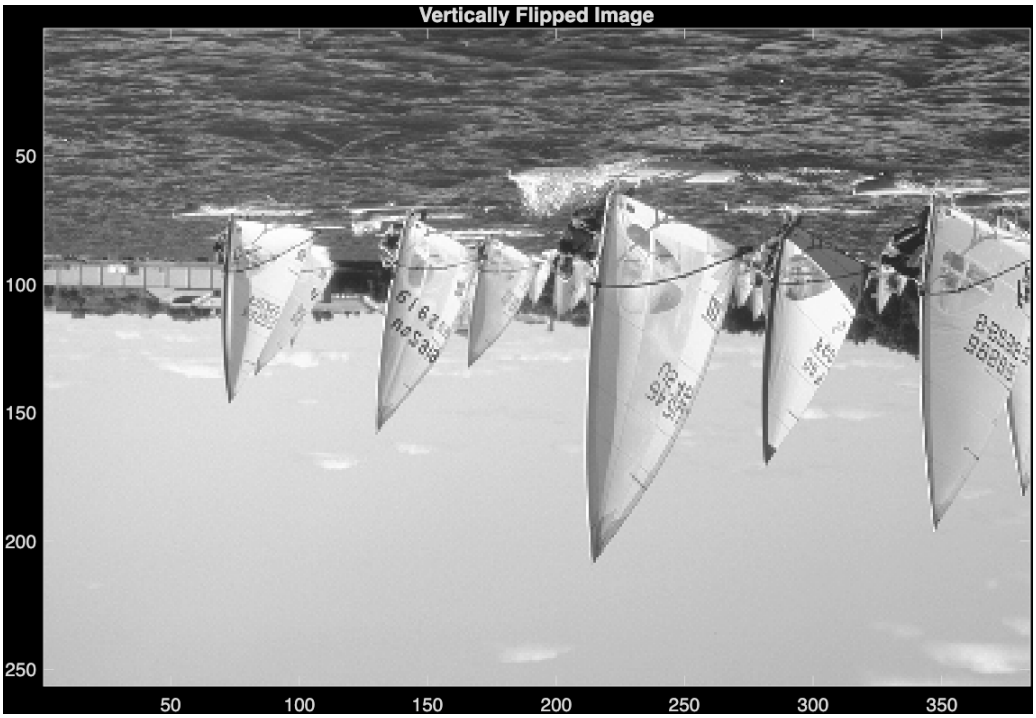

Figure 3: Vertically Flipped Yacht Image

Figure 4: Negated Yacht Image

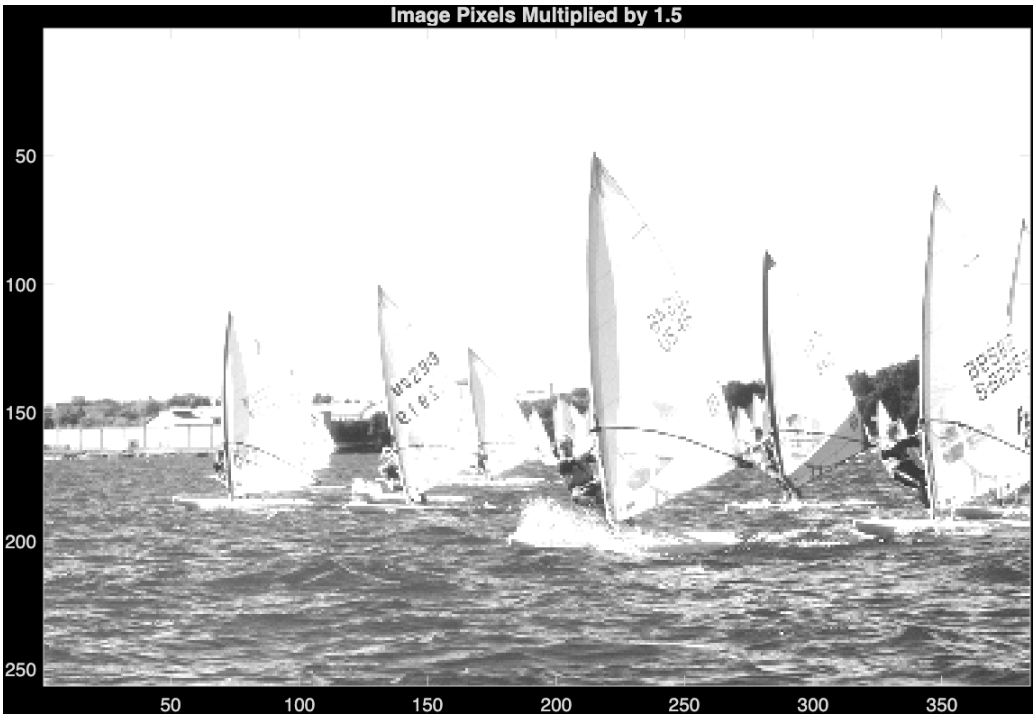

Figure 5: Yacht Image Pixel Intensities Multiplied by 1.5

The effect seen in Fig. 5 is the pixel intensities get close to a value of 255 and the image becomes more white. The reason for this effect is the pixel intensities are multiplied by a factor of 1.5. If the pixel intensities were multiplied by a value less than 1, the picture would appear more black as the image intensities would approach 0 or black. A multiplication of 1 would have no change in the original image.

2 Histogram of an Image

Figure 6: Original House Image

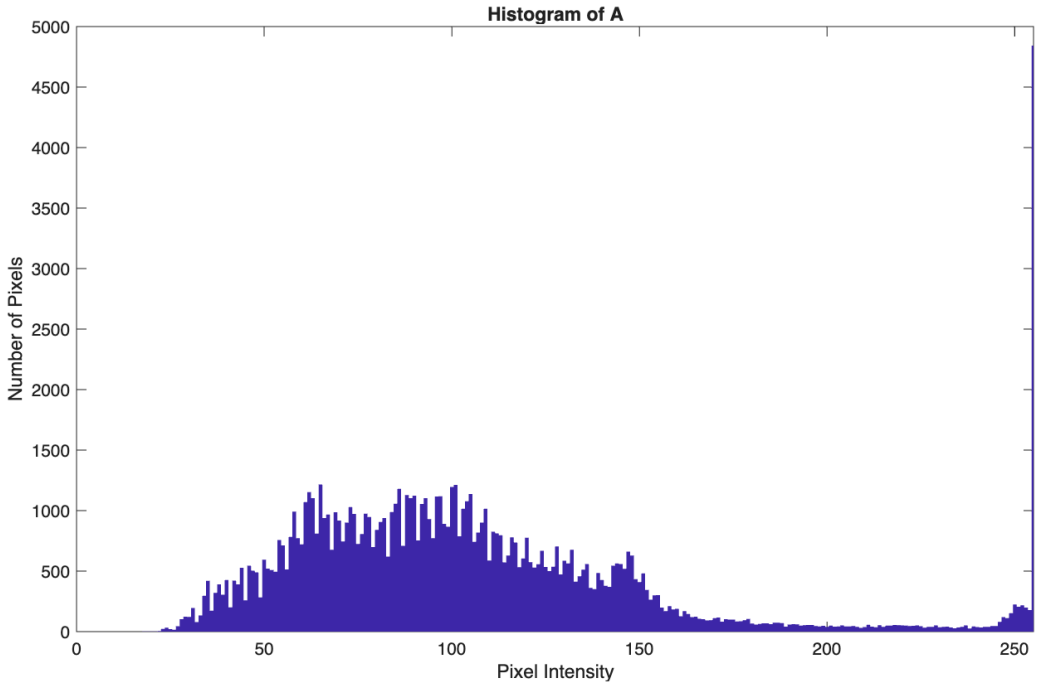

Figure 7: Histogram of House Image

The histogram in Fig. 7 has a primary clumped distributions of pixel intensities. The pixel clump occurs at a range from about 30 to 160 pixel intensities. This range has a consistent number of pixels at about 750 for each pixel intensity.

Additionally, about 4800 pixels have a pixel intensity of 255 (or black).

3 Pointwise Transformations

pointTrans.m

```
% Purpose: Stretch the image from pixels T1 to T2
% Input (input, T1, T2): Input image, Range Values [T1, T2]
% Output (output): Stretched image
function output = pointTrans(input, T1, T2)
    % Find the rows and columns of the input image
    image = double(input);
    [rows, cols] = size(image);

    output = zeros(rows, cols);

    for r = 1:rows
        for c = 1:cols
            pixVal = image(r, c);

            if pixVal < T1
                % Black pixels less than range
                output(r, c) = 0;
            elseif pixVal > T2
                % White pixels more than range
                output(r, c) = 255;
            else
                % Stretch the rest of the values to full range [0, 255]
                output(r, c) = 255 * (pixVal - T1) / (T2 - T1);
            end
        end
    end
end
```

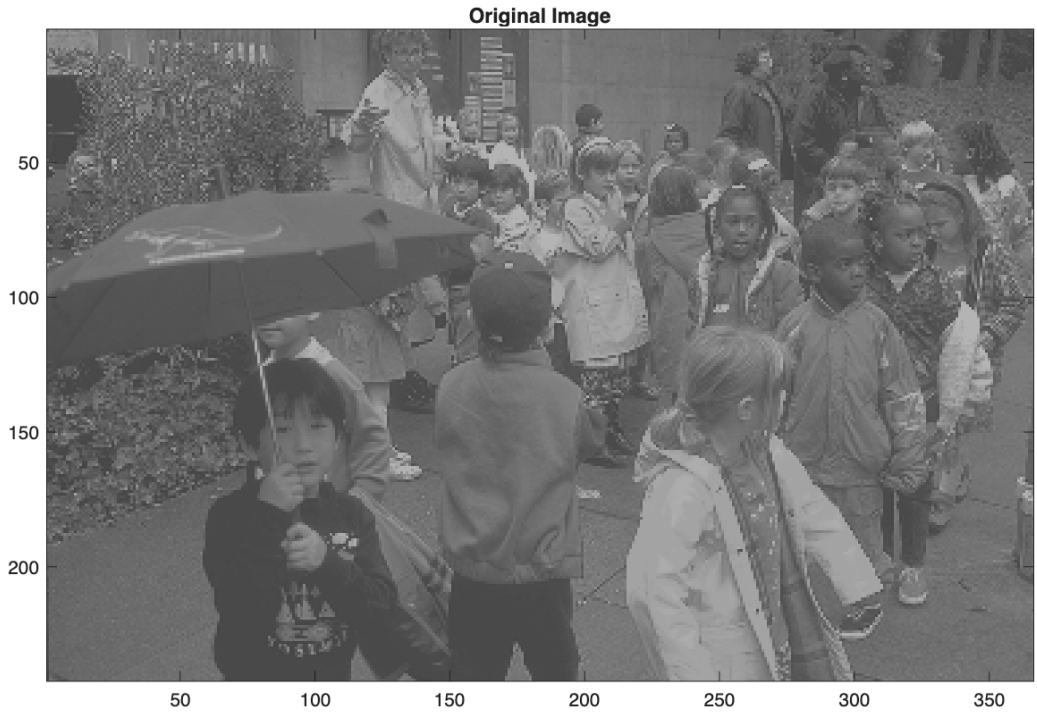

Figure 8: Original Narrow Image

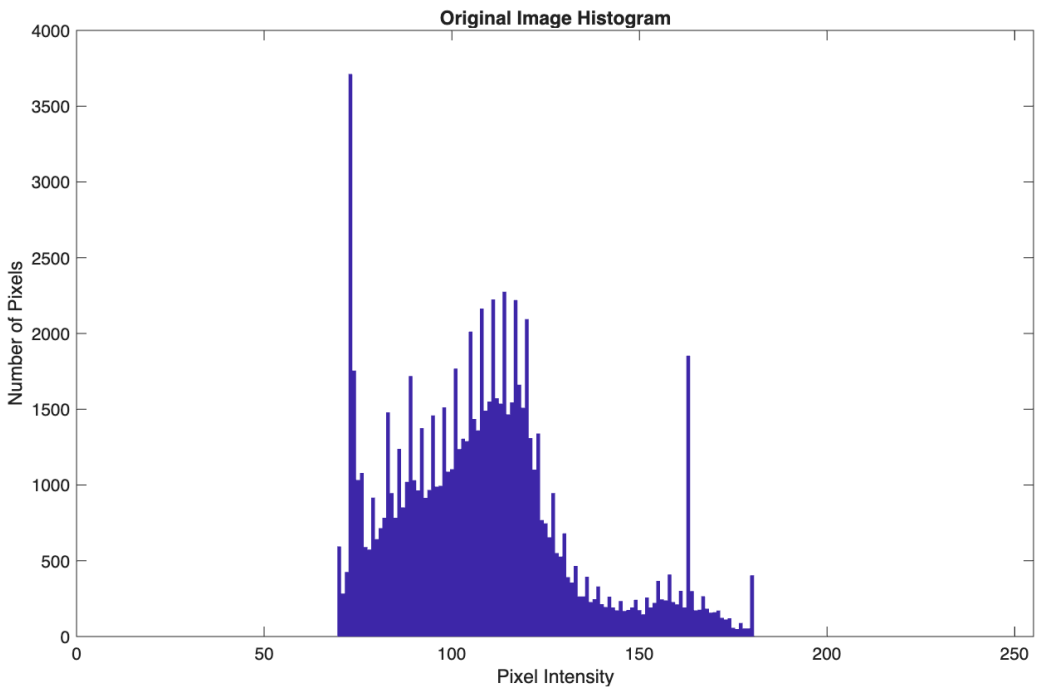

Figure 9: Histogram of Narrow Image

Figure 10: Stretched Narrow Image

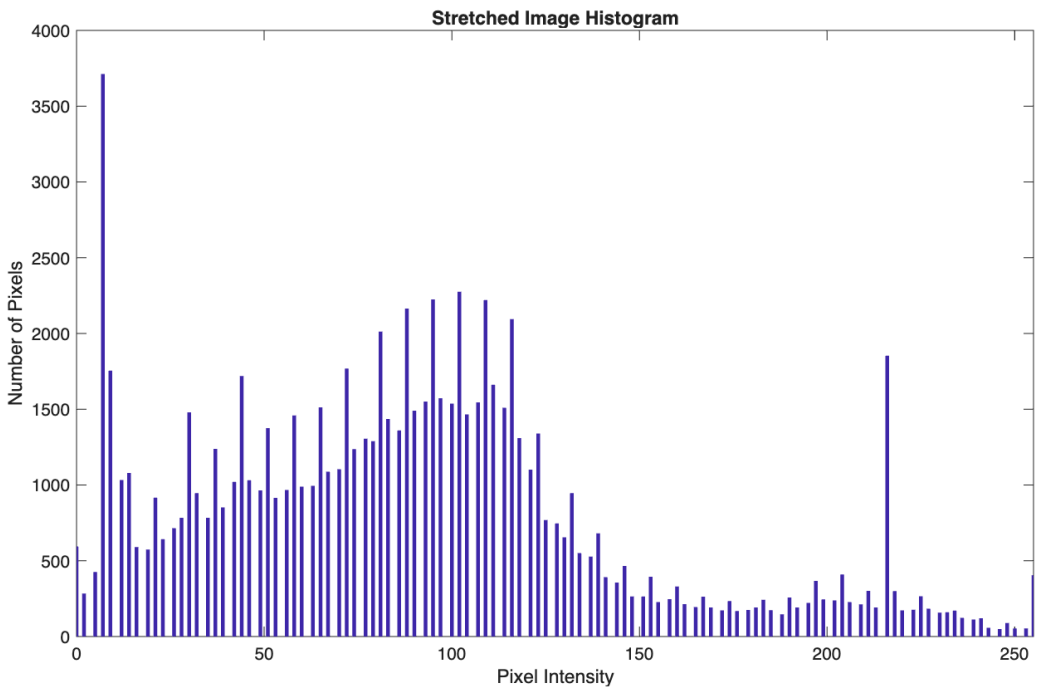

Figure 11: Histogram of Stretched Narrow Image

Stretching an image, as seen in Fig. 10, makes the pixels appear brighter and cleaner as opposed to the original image in Fig. 8. However, when looked at closely, Fig. 10 is blurry at the outlines of the objects and people due to the stretching effect. Moreover, while Fig. 10 provides a cleaner version than the foggy original version, the stretched image intensifies the individual pixels to make the image more blurry.

The histogram in Fig. 9 is more clumped and confined in a pixel intensity range of about 70 to 180. The histogram in Fig. 11 provides a much more diverse range where the pixel intensities span the entire range. The zeros in the histogram correspond to the black pixels in the image.

4 Gamma Correction

Figure 12: Original Dark Image

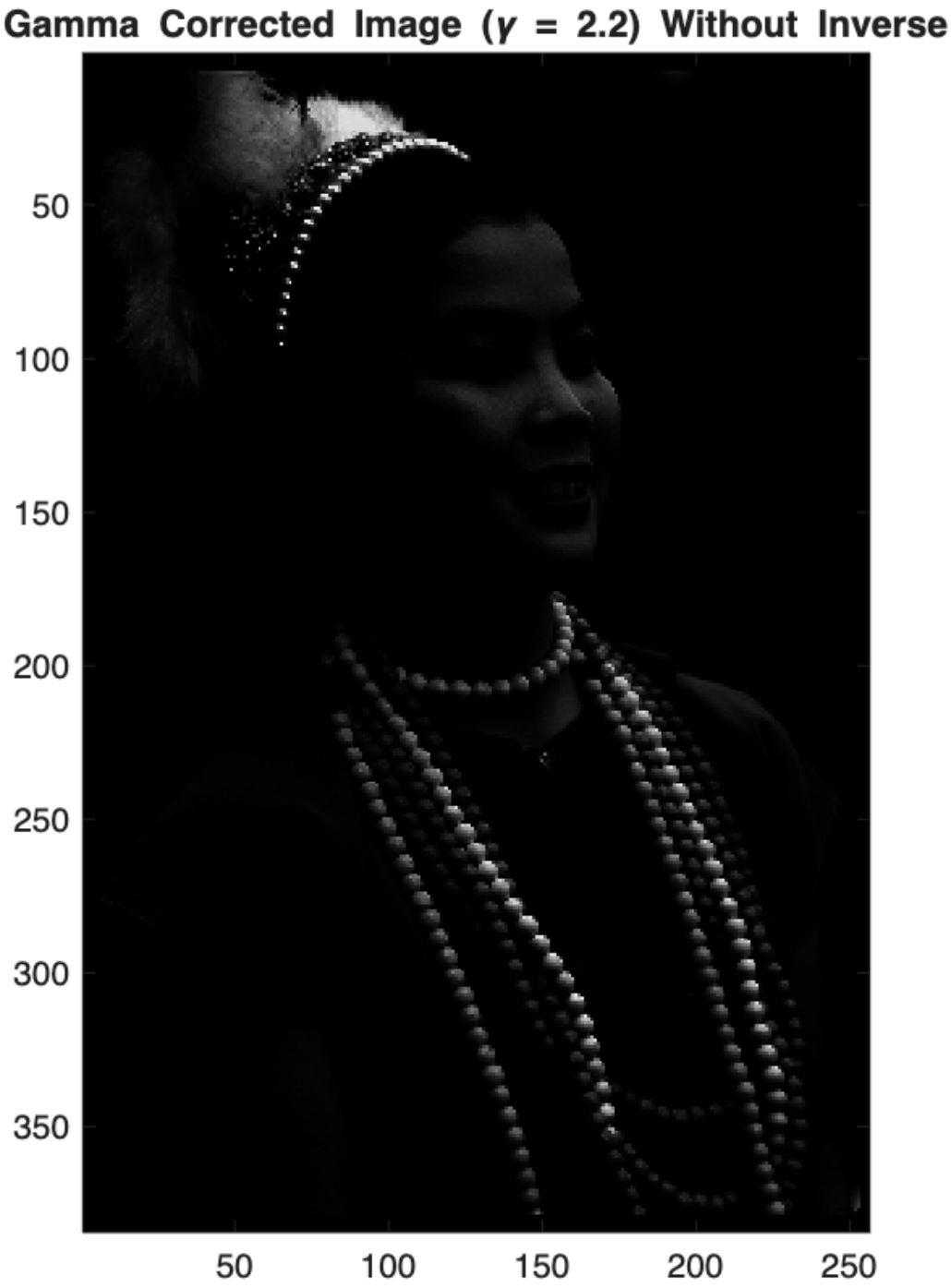

Figure 13: Original Gamma Corrected Image

Figure 14: Inverse Gamma Corrected Image

gammCorr.m

```
% Purpose: Correct an image that is too dark/light
% Input (x, gamma): Original pixel value, gamma value
% Output (y): Pixel Intensity
function y = gammCorr(x, gamma)
    % Inverse of function given in the problem description
    y = 255 * ((double(x) / 255.0) .^ (gamma));
end
```

The gamma corrected image as seen in Fig. 13 with the original equation given in the problem statement was too dark. Therefore, we used the inverse as seen in the Matlab script gammCorr.m to produce a better quality image. Figure 14 shows the inverse gamma corrected image. The inverse gamma corrected image is clearer and brighter compared to the original dark version.

5 Image Smoothing

gaussFilter.m

```
% Purpose: Filter an image using a guassian filter
% Input (N, var): Size of the filter, variance
% Output (h): NxN filter
function h = gaussFilter(N, var)
    % To be centered around origin and be symmetric, N must be odd
    if mod(N, 2) == 0
        error('N must be an odd number');
    end

    % Find distance from origin
    distance = floor(N / 2);

    % Create a meshgrid around center
    [i, j] = meshgrid(-distance:distance, -distance:distance);

    % Calculate Gaussian values with equation given in problem statement
    h = exp(-(i.^2 + j.^2) / (2 * var));

    % Normalize the filter
    h = h / sum(h(:));
end
```

medianFilter.m

```
% Purpose: Filter an image using a median filter
% Input (X): Input Image
% Output (Y): Output Filtered image
function Y = medianFilter(X)
    % Find rows and columns of input image
    [rows, cols] = size(X);
    Y = double(X);
    padding = double(X);

    % Skip borders of image
    for r = 2:(rows - 1)
        for c = 2:(cols - 1)
            % Find 3x3 window around the pixel
            window = padding(r-1:r+1, c-1:c+1);

            % Find the median of the pixels in the window
            Y(r, c) = median(window(:));
        end
    end
end
```

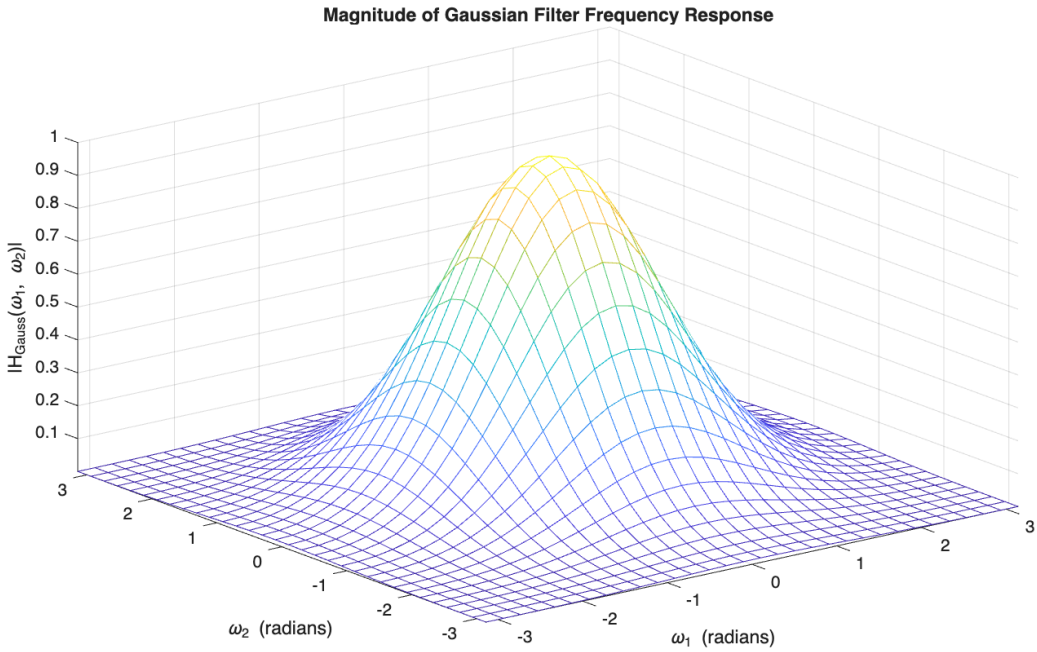

Figure 15: Magnitude of the Frequency Response of the Gaussian Filter

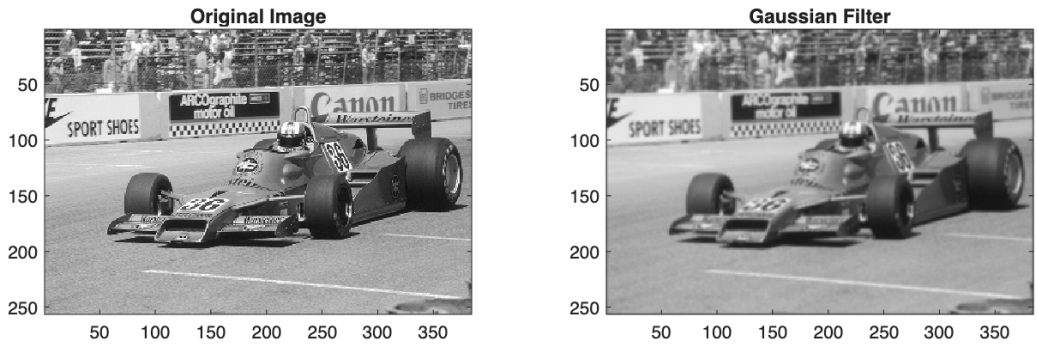

Figure 16: Original Race Image and Gaussian Filter

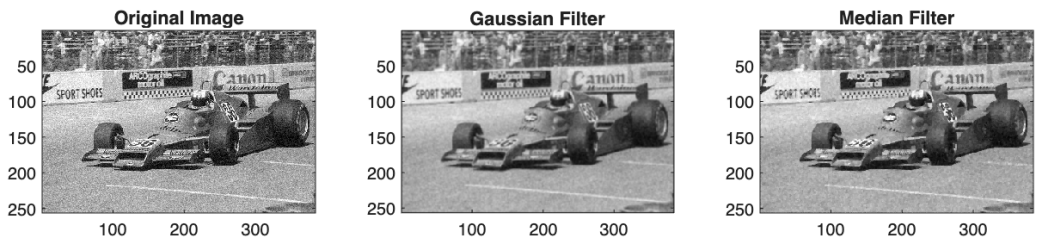

Figure 17: Noise1 with Additive White Gaussian Noise

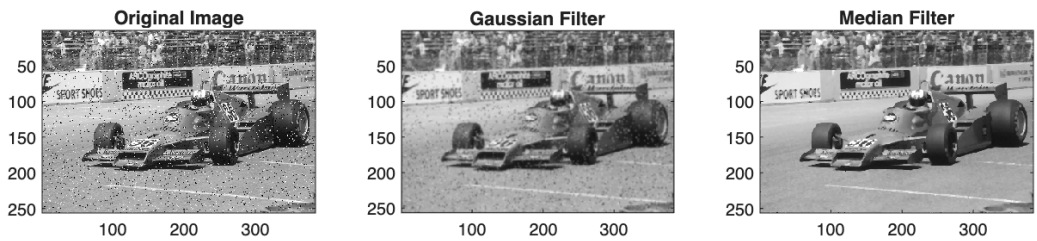

Figure 18: Noise2 with Salt and Pepper noise

The filters in Fig. 17 with additive white Gaussian noise both smooth out the image. The Gaussian filter smooths the image at a stronger degree where every outline of a shape is clear. The median filter, however, is more pixelated compared to the Gaussian filter. In both images, differences between the most intense black and white pixels appears to be smaller.

The "salt and pepper" noise in Fig. 18 covers the image with black and white pixels. The median filter does a good job at smoothing the image and making the features appear clearer. However, when zooming in, the car and other objects are distorted with "filler" pixels. Additionally, the Gaussian filter morphs the dots to gray pixels that are still visible but try to blend in with the picture.

6 Image Sharpening

The provided equation from the problem statement,

$$g(i,j) = \alpha f(i,j) - \beta[f(i,j) * *h(i,j)] \tag{1}$$

can be translated into the frequency domain:

$$G(w_1,w_2) = \alpha F(w_1,w_2) - \beta[F(w_1,w_2) \cdot H(w_1,w_2)] \tag{2}$$

$$G(w_1,w_2) = F(w_1,w_2)(\alpha - \beta H(w_1,w_2)) \tag{3}$$

$$\frac{G(w_1,w_2)}{F(w_1,w_2)} = \alpha - \beta H(w_1,w_2) \tag{4}$$

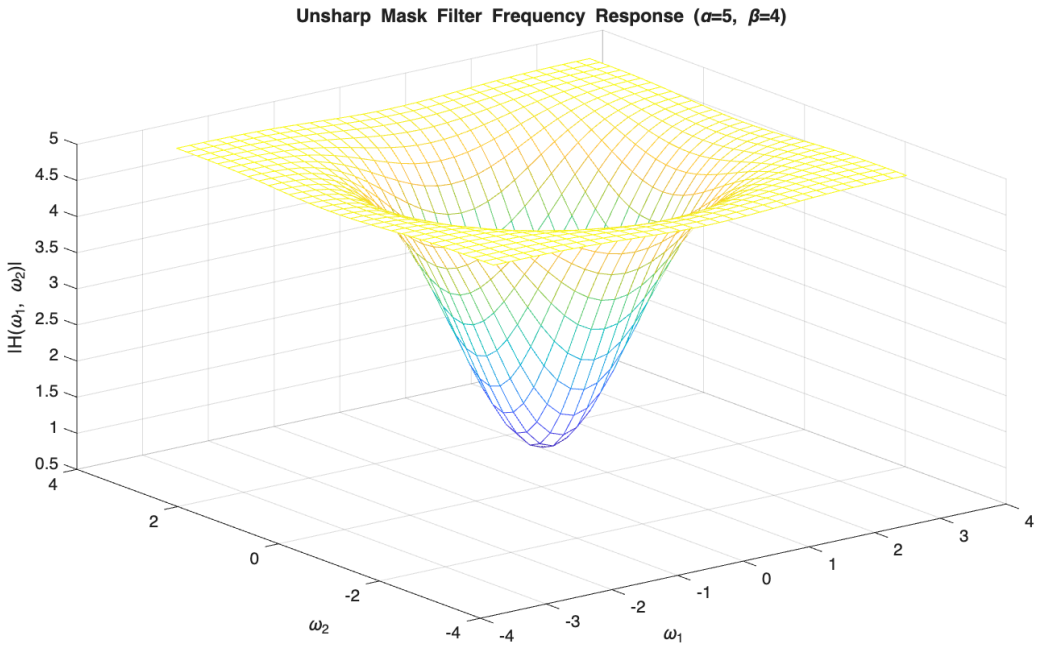

Figure 19: Unsharp Magnitude Response

The plot in Fig. 19 is similar to the provided plots in Figure 16.10 of the problem statement in shape. While Fig. 19 does not have hyperbolic paraboloidic tendencies with curves near the borders of the range, the general structure with a peak at the origin is similar. Specifically, Fig. 19 behaves like Figure 16.10b, where the function is inverted along the z-axis.

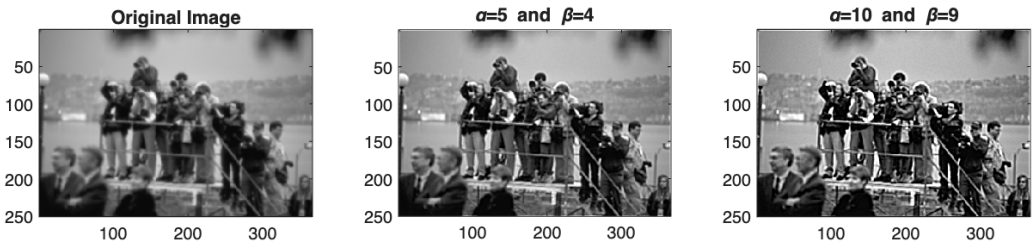

Figure 20: Processed Blur Images

The sharp contrast with the higher values of α and β outline the objects to present a clearer picture. However, with higher values, the image might be oversaturated. Moreover, the lighter pixels in the image with higher pixels intensify and give the picture an unnatural appearance. Whereas, the lower values provide a more real, filtered version of the blurred image.